

KARAR BELGESİ · 2026-09-09

Ajan İletişim Ağı — Seçenekler ve Karşılaştırma

Ajanlar birbirleriyle nasıl konuşacak? Dört taşıma seçeneği, artıları ve eksileriyle.

İçindekiler

0. Sorunun özeti

1. Önce ayırım: protokol ile taşıma aynı şey değil

Neden protokol dosya olmalı

Mesaj yolunun somut hâli

2. Dört taşıma seçeneği

Seçenek A — Subagent (tam otomatik)

Seçenek B — Gerçek sohbetler + oturumlar arası mesajlaşma

Seçenek C — Karma

Seçenek D — Deterministik orkestrasyon betiği

3. Halüsinasyonu sıfırlayan dört kural

4. Tasarıma eklenmesi gereken iki rol

Ortam Ajanı

Hakem

5. Karşılaştırma tablosu

6. Öneri

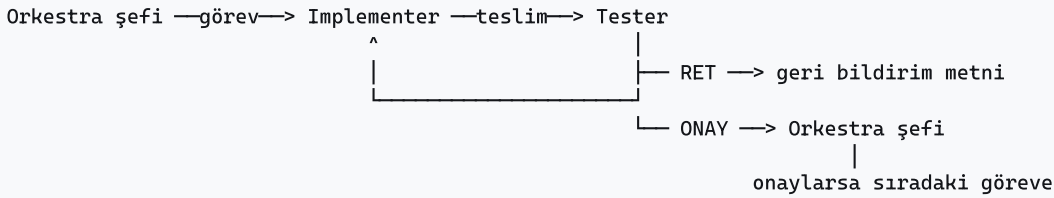
7. Yarın karara bağlanacaklar

8. Karar kaydı

Tarih	2026-09-09
Durum	Karar bekliyor — yarın konuşulacak
Bağlam	Suflör tasarım dokümanı §8'in yeniden yazımı
Karar verilmiş	Görev granülerliği = test-birimi
Karar bekleyen	Taşıma mekanizması (bu belgenin konusu)

0. Sorunun özeti

Tasarım şu şekilde işleyecek:



Her görevin kendi implementer'ı ve kendi tester'ı olacak. Suflör için bu yaklaşık **70-90 görev + 70-90 tester** demek.

Sorulan soru şuydu: "Bu ajanlar arasında bir iletişim ağı kurabilir miyiz, yoksa her seferinde mesajı ben mi taşıyacağım?"

Cevap: mesaj taşımayacaksınız. Bunun dört yolu var ve bu belge dördünü karşılaştırıyor.

1. Önce ayrım: protokol ile taşıma aynı şey değil

Bu ayrımı yapmazsak tartışma karışıyor.

Protokol = mesajın *biçimi ve nerede durduğu*. Görev paketi neye benzer, teslim raporu ne içerir, geri bildirim nasıl yazılır, kanıt nerede saklanır.

Taşıma = mesajın *nasıl iletildiği*. Ajanlar birbirine mi yazar, ortak bir dizinden mi okur, bir betik mi çağırır.

Protokol kararı verilmiş sayılmalı: **protokol dosyadır**. Taşıma ise açık soru ve bu belgenin asıl konusu.

Neden protokol dosya olmalı

Halüsinasyon sıfır hedefi denetim izi gerektiriyor. Sohbet mesajı kaydırma geçmişinde kaybolur. Dosya kalıcıdır: sonradan diff'lenir, yeniden okunur, "bu ajan gerçekten ne iddia etmişti" sorusu kanıtlarla cevaplanır. Denetlenemeyen bir sistemde halüsinasyon sıfırlanamaz, sadece görünmez olur.

Tester körlüğü ancak dosyayla sağlanır. Tester'ın implementer'ın raporunu okumaması gerekiyor (sebebi §3'te). Hangi ajanın neyi okuyabildiğini dosya düzeniyle kontrol edersin. Sohbet akışında bağlam sızar ve körlük iddiadan öteye geçmez.

Oturumlar kapanır, dosyalar kapanmaz. Bir ajan çökerse, bilgisayar kapanırsa, oturum zaman aşımına uğrarsa iş diskte durur ve kaldığı yerden devam eder.

Mesaj yolunun somut hâli

```
.agents/  
  charters/          rol tanımları (şef yeni rol eklerse buraya)  
  tasks/  
    T-042/  
      packet.md      şef      → implementer  (ne yapılacak)  
      env.md          ortam aj. → tester      (nasıl koşulur)  
      delivery.md     impl.    → şef          (tester OKUMAZ)  
      evidence/       ham komut çıktıları (kanıt deposu)  
      verdict.md      tester   → şef          (onay/ret + kanıt)  
      feedback.md     tester   → implementer  (ret hâlinde)  
      decision.md     şef      → nihai karar  
      round.json      tur sayacı / kilitleme koruması  
  state/  
    board.json        tüm görevlerin durumu
```

2. Dört taşıma seçeneği

Seçenek A — Subagent (tam otomatik)

Orkestra şefi tek bir oturumdur. Implementer ve tester'ları kendi içinde alt-ajan olarak başlatır; işleri bitince sonuç otomatik döner. Tüm mesajlaşma `.agents/` altında dosya olarak birikir.

Artıları

- Hiçbir mesaj taşımazsın, hiçbir oturum açmayı unutmazsın
- Kapanan oturum, kopan zincir sorunu yok
- Paralellik kolay: bağımsız görevler aynı anda koşar
- Her şeyin dosya izi kalır; istediğin an `.agents/` altını okursun
- En düşük el emeği

Eksileri

- Ajanlar ayrı sohbet olarak durmaz; sonradan "şu ajanla konuşayım" diyemezsin, sadece dosya izlerini okursun
- Alt-ajanın düşünme süreci canlı izlenemez, yalnızca sonucu ve kanıtı görürsün
- Tek oturum çökerse o anki koordinasyon kaybolur (dosyalar durduğu için iş kaybolmaz)

Ne zaman doğru: İş hacmi yüksek ve tekrarlıysa, izlemekten çok bitirmek istiyorsan.

Seçenek B — Gerçek sohbetler + oturumlar arası mesajlaşma

Her ajan senin açtığın ayrı bir sohbetir. Oturumların adresi vardır ve birbirlerine doğrudan mesaj atabilirler.

Artıları

- Her ajanın düşünme sürecini canlı izleyebilirsin
- İstedığın ajana istediğin an müdahale edebilirsin

- Bir ajanın sohbeti kalıcıdır; sonradan geri dönüp bağlamıyla devam ettirebilirsin
- Zihinsel model en sezgiseli: "her ajan bir kişi, her sohbet onun masası"

Eksileri

- **Her oturumu sen açmalı ve açık tutmalısın.** 90 görev + 90 tester için bu ciddi bir el emeği
- Kapanan oturum zinciri kırar — bu gerçek bir risk, ölçüldü: mevcut eş oturumların ikisi de kapalı durumda
- Paralellik senin kaç sohbet yönetebildiğinle sınırlı
- Ajan sayısı arttıkça hangi sohbetin ne beklediğini takip etmek başlı başına iş

Ne zaman doğru: Az sayıda, uzun ömürlü, kritik rol varsa. 180 ajan için uygun değil.

Seçenek C — Karma

Uzun ömürlü kritik roller (orquestra şefi, hakem) gerçek sohbet olarak açılır ve izlenir. Kısa ömürlü implementer ve tester'lar alt-ajan olarak koşar.

Artıları

- İzlenebilirlik ile otomasyonu dengeler
- Kritik kararları (onay, ret, yeni rol açma) canlı görürsün
- Hacimli iş otomatik akar, sen sadece kapıları izlersin

Eksileri

- İki mekanizmayı birden yönetmek gerekir
- Hangi rolün hangi mekanizmada olduğu bir karar daha demek; sınır zamanla bulanıklaşır
- Hata ayıklarken "bu mesaj nereden geldi" sorusu iki farklı yerde aranır

Ne zaman doğru: Otomasyonu istiyorsan ama kritik kararları görmeden onaylamak istemiyorsan.

Seçenek D — Deterministik orkestrasyon betiği

Implementer → tester → şef döngüsü bir betik olarak kodlanır. Paralel dallar, şema doğrulamalı ajan çıktıları, yarıda kalan koşuyu kaldığı yerden sürdürme desteği vardır.

Artıları

- **En tekrarlanabilir seçenek.** Aynı betik aynı sonucu verir; süreç yoruma açık değil
- Ajan çıktısı şemayla doğrulanır — biçimsel olarak bozuk cevap kapıdan geçemez, bu doğrudan halüsinasyon savunmasıdır
- Kapı mantığı (kaç tur, ne zaman hakeme git) kodda yazılıdır, ajanın yorumuna bırakılmaz
- En denetlenebilir: süreç kodun kendisi

Eksileri

- **Düzinelerce ajanı aynı anda başlatabilir; token maliyeti yüksek.** Açık onay gerektirir
- Betiği yazmak ve doğrulamak ayrı bir iş
- Esneklik düşük: şef "burada yeni bir rol lazım" derse betiği değiştirmek gerekir — sınırsız rol açma isteğiyle doğrudan çelişir
- Hata ayıklaması diğerlerinden zor

Ne zaman doğru: Süreç oturduktan ve tekrar tekrar koşacak hâle geldikten sonra.

3. Halüsinasyonu sıfırlayan dört kural

Taşıma seçeneğinden bağımsız olarak bunlar geçerli. Ajan sayısını artırmak tek başına halüsinasyonu **azaltmaz, çoğaltır** — her ajan yeni bir uydurma yüzeyidir. Azaltan şey aşağıdaki kapılardır.

1 · Ajanın sözü delil değildir. "Testler geçiyor" cümlesi hiçbir şey ifade etmez. Yalnızca çalıştırılmış komut ve ham çıktı kabul edilir. Çıktı yoksa iş teslim edilmemiştir.

2 · Tester kör çalışır. Tester, implementer'ın teslim raporunu okumaz. Yalnızca görev paketini (ne yapılmalıydı) ve kodu (ne var) görür; testleri kendi yazar ve kendi koşar. Raporu okursa ona demirler ve aynı yanlışlığı onaylar. Bağımsızlık ancak körlükle sağlanır.

3 · Sözleşmeler makineyle denetlenir. Tip denetimi, şema doğrulama, arayüz uyum testleri. Ajan yorumuna bırakılan sınır yok.

4 · Tester de delil zorunda. Tester "bu bozuk" diyemez; yeniden üretilebilir komut ve hata çıktısı vermek zorundadır. Aksi hâlde tester'ın halüsinasyonu implementer'ı kilitler. Bu kural çoğu tasarımda unutuluyor ve sistemi tıkayan şey oluyor.

4. Tasarıma eklenmesi gereken iki rol

Anlattığın akışta iki boşluk var; ikisi de sistemi kilitleyebilecek türden.

Ortam Ajanı

Tester'ların koşum ortamını **bir kez** kurar: sahte implementasyonlar, fixture'lar, veri örnekleri, izole çalışma dizini, koşum komutları. Çıktısı her görevin `env.md` dosyasıdır.

Olmazsa her tester dünyayı baştan kurar: yavaş, tutarsız, ve her tester farklı bir ortamda test ettiği için sonuçlar kıyaslanamaz.

Hakem

Implementer ile tester belirli sayıda turda (öneri: 3) anlaşamazsa devreye girer. İkisinin kanıtına bakar, hangisinin haklı olduğuna karar verir, kararını gerekçesiyle yazar.

Olmazsa sonsuz ping-pong olur: implementer "düzelttim" der, tester "hâlâ bozuk" der, döngü kapanmaz ve görev asla bitmez. Bu, çok ajanlı sistemlerin en sık takıldığı yerdir.

5. Karşılaştırma tablosu

	A · Subagent	B · Gerçek sohbetler	C · Karma	D · Betik
Senin el emeğin	Yok	Çok yüksek	Düşük	Yok (kurulumdan sonra)
Canlı izlenebilirlik	Düşük	Yüksek	Orta-yüksek	Düşük
Kalıcı denetim izi	Yüksek (dosya)	Yüksek (dosya)	Yüksek (dosya)	En yüksek
Paralellik	Yüksek	Düşük	Yüksek	En yüksek
Kopma riski	Düşük	Yüksek	Düşük	Düşük
Tekrarlanabilirlik	Orta	Düşük	Orta	En yüksek
Sınırsız yeni rol	Kolay	Kolay	Kolay	Zor
Token maliyeti	Orta	Orta	Orta	Yüksek
Kurulum eforu	Düşük	Düşük	Orta	Yüksek

6. Öneri

Seçenek A ile başla, süreç oturunca D'ye geç.

Gerekçe: 180 ajanlık bir iş yükünde B'nin el emeği gerçekçi değil ve kopma riski en yüksek olan seçenek o. D ise doğru varış noktası ama şu an erken — süreci daha bir kez bile çalıştırmadık, neyin işe yaradığını bilmiyoruz, ve "orquestra şefi gerekirse sınırsız yeni rol açabilsin" isteğinle doğrudan çelişiyor. Betik esnekliği öldürür; esnekliğe şu aşamada ihtiyaç var.

A'yı seçmek D'yi kaybettirmez: protokol dosya olduğu için, süreç oturduktan sonra aynı dosya düzenini bir betikle sürmek küçük bir iş olur. Tersı doğru değil — betikle başlayıp esnekliğe dönmek yeniden yazım demektir.

C, A'ya sonradan eklenebilir: orkestra şefini ayrı bir gerçek sohbete taşımak tek satırlık bir karardır.

7. Yarın karara bağlanacaklar

- Taşıma seçeneği** — A / B / C / D
- Kilitlenme eşiği** — kaç turdan sonra hakeme gidilir (öneri: 3)
- Ortam Ajanı kapsamı** — alan başına mı (capture, ocr, translate, store, ui) yoksa tek merkezî ajan mı
- Şefin yeni rol açma yetkisinin sınırı** — sınırsız isteniyor; sonsuz özyinelemeyi (ajan açan ajan açan ajan) engelleyecek kural ne olacak. Öneri: yeni rol yalnızca şef tarafından açılabilir, her rol bir tüzük (charter) + sahiplik dizini + kabul kriteri ile gelir
- Paralellik tavanı** — aynı anda en fazla kaç görev koşsun
- Bütçe** — 180 ajanlık bir koşunun token maliyeti kabul edilebilir mi, yoksa görev sayısı düşürülmeli mi

8. Karar kaydı

Karar	Seçim	Gerekçe
Görev granülerliği	Test-birimi	Bağımsız kabul testi yazılabilen en küçük iş. Maksimum granülerlik verir ama doğrulanamaz parça üretmez
Mesaj protokolü	Dosya	Denetim izi, tester körlüğü, oturum bağımsızlığı
Her göreve tester	Evet	Kullanıcı kararı; kapı mantığının temeli
Tester'ın kanıt zorunluluğu	Evet	Tester halüsinasyonu implementer'ı kilitlemesin
Taşıma	<i>karar bekliyor</i>	Bu belge